

Guia Definitivo do QA Pleno em APIs de Pagamento

Operação em Produção e Staging: Automação Playwright, Banco de Dados Relacional, Observabilidade, Auditoria de PRs e Plano de Ação de 60 Dias

Função no Time: QA Pleno • SDET

Domínio: Gateways, PSPs, Pix v2, Adquirência & PCI-DSS

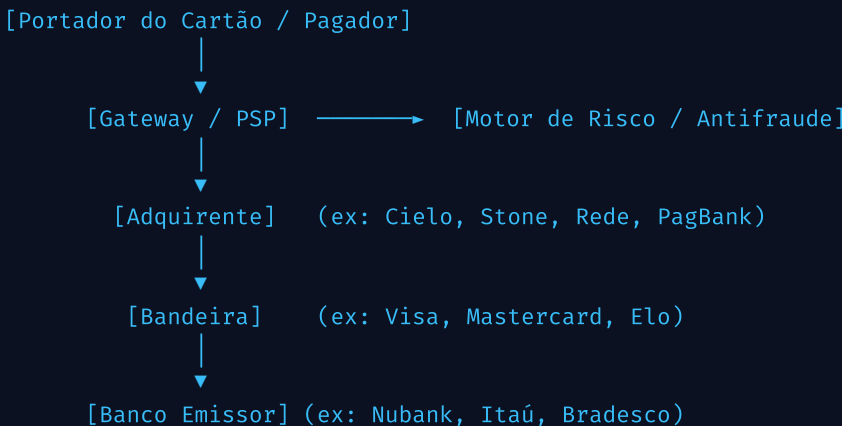
Stack Central: Playwright, TypeScript, PostgreSQL, Docker, Toxiproxy

Foco de Atuação: Blindagem de CI/CD, Não-Regressão e Integridade Contábil

1. Fundamentos do Domínio de Pagamentos & Precisão Numérica

Em sistemas financeiros, a automação de testes não se limita a asserções de status HTTP 200. Falhas lógicas em APIs de pagamento geram passivos contábeis diretos, perdas de liquidação, vulnerabilidades a fraudes e severas sanções aplicadas pelo Banco Central do Brasil (BACEN) e pelo PCI Security Standards Council.

1.1 Os Atores do Arranjo de Pagamentos

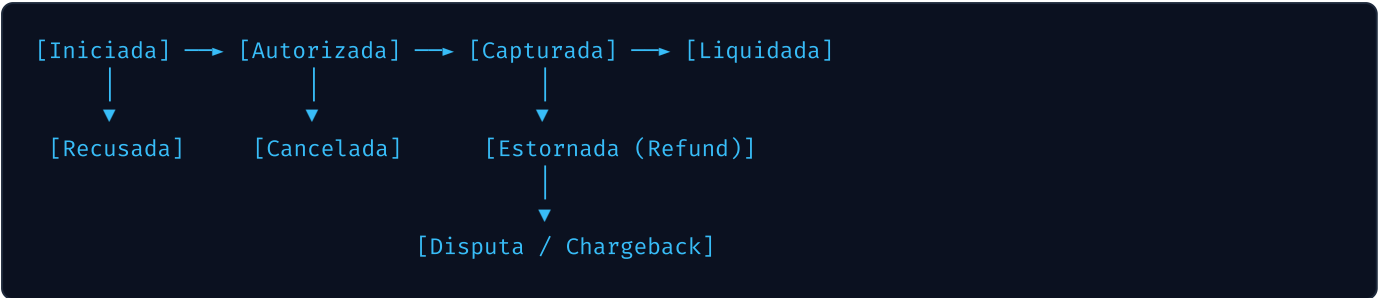


- **Gateway:** Integrador puramente técnico que recebe os dados da aplicação web/mobile, tokeniza, criptografa e distribui às pontas adquirentes sem liquidar valores.

- **PSP (Payment Service Provider):** Plataforma agregadora que unifica gateway, conciliação e liquidação financeira sob uma mesma licença contratual (ex: Pagar.me, Asaas, Stripe).
- **Adquirente:** Entidade credenciada pelas bandeiras para habilitar estabelecimentos comerciais e processar a captura e liquidação financeira das transações.
- **Bandeira:** Define as regras operacionais globais de segurança, roteamento, compensação e limites das transações.
- **Emissor:** Instituição financeira que concede o limite de crédito ou custodia a conta corrente do comprador, aprovando ou recusando a transação conforme saldo e risco comportamental.

1.2 Máquina de Estados Finitos (FSM) da Transação

O ciclo de vida de uma cobrança com cartão é governado por uma **Máquina de Estados Finitos** com transições determinísticas. Um QA precisa testar exaustivamente não apenas as transições válidas, mas também a rejeição de transições inválidas (ex: tentar capturar uma transação já cancelada):



Operação	Definição Técnica	Efeito Contábil no Sistema
Autorização (Auth)	Valida a integridade do cartão, limite disponível e aprovação no emissor.	Bloqueia o limite do portador. <i>Nenhum dinheiro trocou de mãos ainda.</i>
Captura (Capture)	Confirmação da transação pelo lojista para efetivação contábil do débito.	Lança a compra na fatura do cliente e programa o recebível futuro do lojista.
Cancelamento (Void)	Anulação da reserva de saldo antes do fechamento diário do lote bancário.	Desfaz a pré-autorização imediatamente sem cobrança na fatura.
Estorno (Refund)	Devolução do valor após a transação já ter sido capturada e consolidada.	Dispara a liquidação reversa entre emissor e adquirente; pode ser total ou parcial.

1.3 O Perigo do Ponto Flutuante (IEEE 754) em Sistemas Financeiros

Processadores modernos representam frações decimais na memória através da norma binária IEEE 754. Operações triviais como `0.1 + 0.2` resultam na dízima binária finita `0.30000000000000004`. Em gateways que processam milhões de transações por hora, pequenas discrepâncias fracionárias acumulam rombos contábeis massivos.

Regra Fundamental para Automação de APIs

APIs de pagamento devem operar com inteiros na menor fração da moeda (ex: `amountInCents: 1000` para R\$ 10,00) ou strings numéricas de precisão fixa. O framework de testes do QA deve validar que payloads contendo números decimais do tipo `10.99` sejam prontamente rejeitados com `400 Bad Request` ou `422 Unprocessable Entity`.

2. Redes, Protocolo HTTP & Rastreabilidade Distribuída

Para atuar com autonomia no dia a dia, o QA deve dominar a camada de transporte, a semântica rigorosa dos códigos HTTP e como auditar microsserviços via rastreamento distribuído.

2.1 O Ciclo Físico de uma Chamada HTTP

```
[Playwright Test Process]
| 1. Resolução DNS (api.pagamentos.com → 192.0.2.1)
▼
[TCP 3-Way Handshake] (SYN → SYN-ACK → ACK)
| 2. Handshake Criptográfico TLS 1.3 (Troca de chaves Diffie-Hellman, certificados mTLS)
▼
[Envio da Requisição HTTP Criptografada]
| POST /v1/charges HTTP/1.1
| Host: api.pagamentos.com
| Authorization: Bearer eyJhbGciOi...
| Idempotency-Key: b3c1-4820-91a1
| X-Correlation-ID: 7a9e34b1-8409-42b8-9311-bfad6ce2e104
| Content-Type: application/json
|
| {"amountInCents": 10000, "currency": "BRL"}
▼
[API Gateway / Proxy Reverso (Kong / Nginx / ALB)]
| Validação mTLS, Rate Limiting, Roteamento
▼
[Microsserviço de Pagamento]
```

2.2 Matriz de Status Codes e Semântica de Negócio

Status Code	Cenário Semântico em Pagamentos	Conduta Correta do QA na Automação
400 Bad Request	Payload mal formatado, sintaxe JSON corrompida ou tipos de dados inválidos.	Asserir quando tipos primitivos estiverem violados (ex: string em campo numérico).
422 Unprocessable Entity	Sintaxe válida, mas violação de regra de negócio (ex: data de expiração no passado, estorno maior que saldo).	Asserir mensagem de erro amigável e código de erro de domínio específico.
401 Unauthorized	Token ausente, inválido ou expirado.	Validar que nenhum detalhe de infraestrutura vaze no body da resposta.
403 Forbidden	Autenticado com sucesso, mas sem autorização de acesso ao recurso solicitado.	Testar cenários de permissão e isolamento de tenants (Lojista A acessando recurso B).
409 Conflict	Conflito de estado do recurso (ex: reutilizar Idempotency-Key com payload modificado).	Validar que a transação original seja preservada intacta sem alteração de valores.
502 / 504 Gateway Error	Instabilidade ou timeout ao conectar com a adquirente ou autorizador externo.	Validar que o gateway ative políticas de retentativa, fallback ou circuito aberto (Circuit Breaker).

2.3 Rastreabilidade Distribuída (Distributed Tracing)

Em ambientes de microsserviços, quando um pagamento falha, o QA precisa auditar exatamente onde o fluxo foi interrompido. Toda requisição enviada pelo framework de testes automatizados deve injetar headers de correlação padronizados:

- `X-Correlation-ID` ou `X-Request-ID` : Identificador único UUIDv4 que deve ser propagado por todos os microsserviços internos e retornado no cabeçalho de resposta HTTP.
- `traceparent` (Padrão W3C Trace Context): Permite correlacionar spans de observabilidade no OpenTelemetry, Datadog ou Jaeger.

3. Autenticação, Tokens & Segurança (OWASP API Top 10)

3.1 Anatomia e Vulnerabilidades de JWT em APIs Financeiras

JSON Web Tokens operam no modelo *stateless* estruturados em `HEADER.PAYLOAD.SIGNATURE` :

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJtZXJjaGFudF8xMjMiLCJyb2xlcYI6WyJjcmVkaXRfY2FyZD9jaGFyZ2UiXSwiZXhwIjoxNzI2MzM1NjAwfQ.Gk9YQ8ZvxG-W_n3J1Z8N3W5Xy98P_b7rK1h3Vw5X0Lk
```

Atenção Crítica aos Riscos de Criptografia

Base64 não é criptografia. Qualquer agente intermediário decodifica o payload de um JWT em milissegundos. É estritamente proibido incluir CVV, senhas ou dados bancários em claims de tokens.

3.2 Matriz de Testes OWASP API Security para Pagamentos

Vulnerabilidade OWASP	Cenário no Domínio de Pagamentos	Estratégia de Teste Automatizado com Playwright
BOLA / IDOR (API1:2023)	Lojista A tenta consultar (<code>GET /v1/charges/{id}</code>) ou estornar transações pertencentes ao Lojista B.	Criar transação com credenciais do Lojista A e tentar acessá-la via token do Lojista B. Esperado: <code>403 Forbidden</code> ou <code>404 Not Found</code> .
Broken Authentication (API2:2023)	API aceita tokens expirados, tokens assinados com o algoritmo <code>none</code> ou assinaturas adulteradas.	Gerar tokens forjados com segredos inválidos e validar rejeição obrigatória com status <code>401 Unauthorized</code> .
Mass Assignment (API6:2023)	Injeção de campos internos privilegiados no payload de criação (ex: <code>{"status": "CAPTURED", "fee": 0}</code>).	Enviar campos administrativos via DTO. O gateway deve ignorá-los e iniciar o pagamento exclusivamente no estado <code>PENDING</code> .

4. Arquitetura do Framework Playwright com TypeScript

O design pattern central adota o padrão **Service Objects**, desacoplando completamente a mecânica das requisições HTTP da camada de lógica de testes, com injeção de dependência nativa através de **Fixtures**.

4.1 Estrutura de Diretórios Padronizada

```
api-automation-framework/
├── .github/workflows/ci.yml      # Pipeline de execução contínua
├── src/
│   ├── clients/                 # Service Objects (camada HTTP)
│   │   ├── BaseClient.ts
│   │   ├── AuthClient.ts
│   │   └── PaymentClient.ts
│   ├── database/                # Módulo de Banco de Dados
│   │   ├── dbConnection.ts      # Pool de conexões PostgreSQL
│   │   └── queries/              # Consultas SQL reutilizáveis
│   ├── factories/               # Geração dinâmica de massa de dados (Faker)
│   │   └── PaymentFactory.ts
│   ├── fixtures/                # Injeção de dependências do Playwright
│   │   └── apiFixtures.ts
│   ├── models/                  # Tipagens estritas e DTOs
│   │   └── payment.types.ts
│   ├── schemas/                 # Contratos em runtime com Zod
│   │   └── payment.schema.ts
│   └── services/                # Recebedores mock para Webhooks e falhas
├── tests/
│   ├── payments/
│   ├── database/                # Testes de auditoria em banco
│   ├── pix/
│   └── webhooks/
├── playwright.config.ts
├── tsconfig.json
└── package.json
```

4.2 Modelagem de Tipos e DTOs (Data Transfer Objects)

```
export type PaymentMethod = 'CREDIT_CARD' | 'PIX' | 'BOLETO';

export type PaymentStatus =
  | 'PENDING'
  | 'AUTHORIZED'
  | 'CAPTURED'
  | 'FAILED'
  | 'VOIDED'
  | 'REFUNDED';

export interface CardDetailsDTO {
  number: string;
  holderName: string;
  expirationMonth: string;
  expirationYear: string;
  cvv: string;
}

export interface SplitRuleDTO {
  recipientId: string;
  percentage: number;
  chargeProcessingFee: boolean;
  liableForChargeback: boolean;
}

export interface CreatePaymentDTO {
  amountInCents: number; // Inteiro: R$ 10,00 = 1000 centavos
  currency: 'BRL' | 'USD';
  customerId: string;
  paymentMethod: PaymentMethod;
  card?: CardDetailsDTO;
  splitRules?: SplitRuleDTO[];
  metadata?: Record<string, string>;
}

export interface RefundRequestDTO {
  amountInCents: number;
}
```

4.3 Service Object Pattern: BaseClient e PaymentClient

```
import { APIRequestContext, APIResponse } from '@playwright/test';
import { randomUUID } from 'node:crypto';

export abstract class BaseClient {
  protected constructor(
    protected readonly request: APIRequestContext,
    protected readonly baseUrl: string
  ) {}

  protected async get(endpoint: string, headers?: Record<string, string>): Promise<APIResponse> {
    return this.request.get(`${this.baseUrl}${endpoint}`, {
      headers: {
        'X-Correlation-ID': randomUUID(),
        ...headers,
      },
    });
  }

  protected async post(
    endpoint: string,
    data: unknown,
    headers?: Record<string, string>
  ): Promise<APIResponse> {
    return this.request.post(`${this.baseUrl}${endpoint}`, {
      data,
      headers: {
        'Content-Type': 'application/json',
        'X-Correlation-ID': randomUUID(),
        ...headers,
      },
    });
  }
}
```

```
import { APIRequestContext, APIResponse } from '@playwright/test';
import { BaseClient } from '../BaseClient';
import { CreatePaymentDTO, RefundRequestDTO } from '../models/payment.types';

export class PaymentClient extends BaseClient {
  constructor(request: APIRequestContext, baseUrl: string) {
    super(request, baseUrl);
  }

  async createPayment(
    payload: CreatePaymentDTO,
    options?: { idempotencyKey?: string; token?: string }
  ): Promise<APIResponse> {
    const headers: Record<string, string> = {};
    if (options?.idempotencyKey) {
      headers['Idempotency-Key'] = options.idempotencyKey;
    }
    if (options?.token) {
      headers['Authorization'] = `Bearer ${options.token}`;
    }
    return this.post('/v1/payments', payload, headers);
  }

  async capture(id: string, token: string, amountInCents?: number): Promise<APIResponse> {
    return this.post(`/v1/payments/${id}/capture`, { amountInCents }, {
      Authorization: `Bearer ${token}`,
    });
  }

  async refund(id: string, payload: RefundRequestDTO, token: string): Promise<APIResponse> {
    return this.post(`/v1/payments/${id}/refund`, payload, {
      Authorization: `Bearer ${token}`,
    });
  }
}
```

4.4 Injeção de Dependências com Custom Fixtures

```
import { test as base, expect } from '@playwright/test';
import { PaymentClient } from '../clients/PaymentClient';
import { Pool } from 'pg';

type PaymentFixtures = {
  paymentClient: PaymentClient;
  merchantToken: string;
  db: Pool;
};

export const test = base.extend<PaymentFixtures>({
  merchantToken: async ({ request }, use) => {
    const authRes = await request.post(`${process.env.API_BASE_URL}/oauth/token`, {
      data: {
        grant_type: 'client_credentials',
        client_id: process.env.CLIENT_ID,
        client_secret: process.env.CLIENT_SECRET,
      },
    });
    expect(authRes.status()).toBe(200);
    const { access_token } = await authRes.json();
    await use(access_token);
  },

  paymentClient: async ({ request }, use) => {
    const client = new PaymentClient(request, process.env.API_BASE_URL || 'http://localhost:3000');
    await use(client);
  },

  db: async ({}, use) => {
    const pool = new Pool({ connectionString: process.env.DATABASE_URL });
    await use(pool);
    await pool.end();
  },
});

export { expect };
```

5. Cenários Críticos: Idempotência, Concorrência & Estornos

5.1 Idempotência: Replays & Prevenção de Adulteração

O cabeçalho `Idempotency-Key` garante que falhas de rede no cliente não resultem em cobranças duplicadas ao reenviar a mesma transação. Adicionalmente, a reutilização de uma chave pré-existente com corpo adulterado deve resultar em conflito explícito.

tests/payments/idempotency.spec.ts

TypeScript

```
import { test, expect } from '../src/fixtures/apiFixtures';
import { PaymentFactory } from '../src/factories/PaymentFactory';
import { faker } from '@faker-js/faker';

test.describe('Validação de Idempotência em Transações', () => {
  test('deve retornar a mesma transação original sem duplicar cobrança', async ({
    paymentClient,
    merchantToken,
  }) => {
    const idempotencyKey = faker.string.uuid();
    const payload = PaymentFactory.createValidCreditCardPayload();

    // 1ª Requisição
    const res1 = await paymentClient.createPayment(payload, { idempotencyKey, token: merchantToken });
    expect(res1.status()).toBe(201);
    const body1 = await res1.json();

    // 2ª Requisição (Replay idêntico)
    const res2 = await paymentClient.createPayment(payload, { idempotencyKey, token: merchantToken });
    expect([200, 201]).toContain(res2.status());
    const body2 = await res2.json();

    expect(body2.id).toBe(body1.id);
  });

  test('deve retornar 409 Conflict ao reutilizar chave com payload adulterado', async ({
    paymentClient,
    merchantToken,
  }) => {
    const idempotencyKey = faker.string.uuid();
    const original = PaymentFactory.createValidCreditCardPayload({ amountInCents: 5000 });
    const tampered = PaymentFactory.createValidCreditCardPayload({ amountInCents: 9900 });

    await paymentClient.createPayment(original, { idempotencyKey, token: merchantToken });
    const resTampered = await paymentClient.createPayment(tampered, { idempotencyKey, token: merchantToken });

    expect([409, 422]).toContain(resTampered.status());
  });
});
```

5.2 Teste de Concorrência e Double-Spending via Promise.all

Simulação de ataque de corrida (*Race Condition*) disparando múltiplas requisições simultâneas no mesmo milissegundo pelo Event Loop do Node.js:

tests/payments/concurrency.spec.ts

TypeScript

```
import { test, expect } from '../src/fixtures/apiFixtures';
import { PaymentFactory } from '../src/factories/PaymentFactory';

test('não deve permitir cobrança dupla ao disparar requisições concorrentes no mesmo milisseg',
  paymentClient,
  merchantToken,
) => {
  const payload = PaymentFactory.createValidCreditCardPayload();
  const fixedKey = `race-${Date.now()}`;

  // Dispara 5 requisições em paralelo real
  const promises = Array.from({ length: 5 }).map(() =>
    paymentClient.createPayment(payload, { idempotencyKey: fixedKey, token: merchantToken })
  );

  const responses = await Promise.all(promises);
  const statuses = responses.map((r) => r.status());

  // Invariante: Estritamente UMA requisição pode retornar 201 Created
  const creations = statuses.filter((s) => s === 201);
  expect(creations.length).toBe(1);

  // As demais devem ser idempotentes (200) ou bloqueadas por lock concorrente (409)
  const allSafe = statuses.every((s) => [200, 201, 409].includes(s));
  expect(allSafe).toBe(true);
};
```

5.3 Estornos Parciais Cumulativos (Prevenção de Saldo Negativo)

Garante que a soma dos estornos parciais nunca exceda o montante total capturado originalmente.

```
import { test, expect } from '../src/fixtures/apiFixtures';
import { PaymentFactory } from '../src/factories/PaymentFactory';

test('deve permitir estornos até o teto capturado e rejeitar excedente com 422', async ({
  paymentClient,
  merchantToken,
}) => {
  // 1. Cria e captura cobrança de R$ 100,00 (10000 centavos)
  const payload = PaymentFactory.createValidCreditCardPayload({ amountInCents: 10000 });
  const authRes = await paymentClient.createPayment(payload, { token: merchantToken });
  const { id } = await authRes.json();
  await paymentClient.capture(id, merchantToken, 10000);

  // 2. Primeiro estorno parcial: R$ 60,00 (6000 centavos)
  const refund1 = await paymentClient.refund(id, { amountInCents: 6000 }, merchantToken);
  expect(refund1.status()).toBe(200);

  // 3. Segundo estorno parcial: R$ 40,00 (4000 centavos - atinge 100% do saldo)
  const refund2 = await paymentClient.refund(id, { amountInCents: 4000 }, merchantToken);
  expect(refund2.status()).toBe(200);

  // 4. Terceiro estorno de R$ 0,01 → Rejeição obrigatória
  const refund3 = await paymentClient.refund(id, { amountInCents: 1 }, merchantToken);
  expect(refund3.status()).toBe(422);
  const errData = await refund3.json();
  expect(errData.code).toBe('REFUND_AMOUNT_EXCEEDS_AVAILABLE_BALANCE');
});
```

6. Split de Pagamentos & Regras Contábeis de Marketplace

Em arquiteturas de marketplace, a API realiza a partilha dos valores entre múltiplos recebedores (vendedores), retém a comissão da plataforma e atribui quem absorve o MDR (*Merchant Discount Rate*) e o risco de chargeback.

tests/payments/split-payment.spec.ts

TypeScript

```
import { test, expect } from '../src/fixtures/apiFixtures';

test('deve validar integridade contábil em split com comissão e taxa de processamento', async request, merchantToken,
) => {
  const payload = {
    amountInCents: 20000, // R$ 200,00
    currency: 'BRL',
    paymentMethod: 'CREDIT_CARD',
    splitRules: [
      {
        recipientId: 'rec_vendedor_externo',
        percentage: 85, // R$ 170,00
        chargeProcessingFee: false,
        liableForChargeback: true,
      },
      {
        recipientId: 'rec_marketplace_taxa',
        percentage: 15, // R$ 30,00
        chargeProcessingFee: true, // Plataforma absorve a taxa
        liableForChargeback: false,
      },
    ],
  },
);

const res = await request.post('/v1/payments', {
  headers: { Authorization: `Bearer ${merchantToken}` },
  data: payload,
});

expect(res.status()).toBe(201);
const body = await res.json();

// Invariante: O somatório das divisões líquidas não pode exceder o valor bruto da transação
const totalAllocated = body.split.reduce(
  (acc: number, cur: any) => acc + cur.netAmountInCents,
  0
);
expect(totalAllocated).toBeLessThanOrEqual(20000);

const vendor = body.split.find((s: any) => s.recipientId === 'rec_vendedor_externo');
expect(vendor.netAmountInCents).toBe(17000);
});
```

7. Banco de Dados: Do Zero Absoluto à Auditoria de Pagamentos

Para um QA de backend e pagamentos, o banco de dados é a **Fonte Única da Verdade** (*Single Source of Truth*). Não basta que a API responda 200 OK no endpoint; se os dados gravados no banco estiverem incorretos, o dinheiro não fecha no final do dia. Este capítulo explica tudo o que você precisa saber sobre bancos relacionais, mesmo que nunca tenha aberto um terminal de banco na vida.

7.1 O que é um Banco de Dados Relacional?

Pense em um banco de dados relacional (RDBMS como PostgreSQL ou MySQL) como um conjunto de **planilhas superpoderosas e conectadas entre si**, porém com regras de segurança rígidas que impedem corrupção de dados.

BANCO DE DADOS: "fintech_db"

TABELA: "customers" (Clientes)

[id (PK)]	[nome]	[email]	[saldo_centavos]
cust_01	Maria Silva	maria@email.com	50000
cust_02	João Souza	joao@email.com	12000

TABELA: "transactions" (Transações)

[id (PK)]	[customer_id (FK)]	Aponta para "customers.id"	
tx_991	cust_01	R\$ 100,00	AUTHORIZED
tx_992	cust_01	R\$ 40,00	CAPTURED
tx_993	cust_02	R\$ 120,00	FAILED

- **Tabela:** Uma entidade de negócio (ex: `transactions` , `customers` , `refunds`). É a "aba" da planilha.
- **Coluna / Campo:** O atributo que define um tipo específico de dado (ex: `amount_in_cents` só aceita números inteiros).
- **Linha / Registro / Tupla:** O registro individual de um evento ocorrido (ex: os dados da transação específica número 991).
- **Chave Primária (Primary Key - PK):** O identificador único e exclusivo daquela linha (ex: UUID ou ID sequencial). Nunca pode haver dois registros com a mesma PK.
- **Chave Estrangeira (Foreign Key - FK):** O elo que liga uma tabela à outra. Se a transação possui `customer_id = 'cust_01'` , o banco só aceita salvar essa transação se `cust_01` existir na tabela `customers` . Isso se chama *Integridade Referencial*.

7.2 Dicionário de Tipos de Dados Cruciais em Finanças

Tipo SQL	O que armazena?	Uso Correto em Pagamentos
<code>INTEGER / BIGINT</code>	Números inteiros sem vírgula.	Valores monetários em centavos (R\$ 10,00 = 1000). Evita erros de arredondamento.
<code>NUMERIC(15, 2) / DECIMAL</code>	Números com precisão exata de casas decimais.	Usado em relatórios contábeis e taxas percentuais (ex: taxa MDR de <code>2.49%</code>).
<code>VARCHAR(n) / TEXT</code>	Textos e strings alfanuméricas.	IDs de adquirentes, nomes, mensagens de erro e dados mascarados (<code>firstSixDigits</code>).
<code>UUID</code>	Identificadores únicos universais (128 bits).	Padrão da indústria para IDs de transações públicas (ex: <code>c8e5d3f0-8c29 ...</code>).
<code>TIMESTAMP WITH TIME ZONE</code>	Data e hora com fuso horário (UTC).	Obrigatório em pagamentos. Todo log financeiro precisa de precisão em milissegundos UTC.
<code>BOOLEAN</code>	Verdadeiro ou Falso (<code>true / false</code>).	Flags de controle interno (ex: <code>is_captured</code> , <code>is_suspicious_fraud</code>).

7.3 Guia Rápido de Comandos SQL para QA

SQL (*Structured Query Language*) é a linguagem utilizada para consultar e manipular esses dados. Abaixo estão os comandos que você utilizará diariamente como QA:

A. Criação de Tabelas (DDL - Data Definition Language)

schema.sql

SQL

```
-- Criação da tabela de transações com travas de segurança
CREATE TABLE transactions (
  id UUID PRIMARY KEY,
  merchant_id UUID NOT NULL,
  idempotency_key VARCHAR(255) UNIQUE, -- Impede duplicação física
  amount_in_cents BIGINT NOT NULL CHECK (amount_in_cents > 0), -- Não aceita valor negativo
  status VARCHAR(30) NOT NULL,
  card_pan_masked VARCHAR(19), -- Ex: 411111*****1111
  card_cvv VARCHAR(4), -- ATENÇÃO: DEVE SEMPRE SER NULL (PCI-DSS)
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Criação da tabela de estornos vinculada por Chave Estrangeira
CREATE TABLE refunds (
  id UUID PRIMARY KEY,
  transaction_id UUID NOT NULL REFERENCES transactions(id),
  amount_in_cents BIGINT NOT NULL CHECK (amount_in_cents > 0),
  status VARCHAR(30) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

B. Consultas e Filtros Essenciais para Testes (DML - Data Manipulation Language)

consultas_auditoria.sql

SQL

```
-- 1. Buscar transações de hoje com valor maior que R$ 100,00
SELECT id, status, amount_in_cents
FROM transactions
WHERE amount_in_cents >= 10000
      AND created_at > CURRENT_DATE;

-- 2. Auditar se houve violação de segurança PCI (procurar CVV salvo)
SELECT id, card_cvv
FROM transactions
WHERE card_cvv IS NOT NULL; -- O resultado deve retornar ZERO linhas!

-- 3. INNER JOIN: Conferir quanto de uma transação foi estornado somando refunds
SELECT
  t.id AS transacao_id,
  t.amount_in_cents AS valor_original,
  COALESCE(SUM(r.amount_in_cents), 0) AS total_estornado
FROM transactions t
LEFT JOIN refunds r ON t.id = r.transaction_id
WHERE t.id = 'd3b07384-d113-4f40-8b4e-e0ef3e120899'
GROUP BY t.id, t.amount_in_cents;
```

7.4 Como Subir e Visualizar o Banco Localmente (Docker + DBeaver)

Você não precisa instalar o PostgreSQL diretamente no seu sistema operacional. A melhor prática é rodar o banco dentro de um container Docker e utilizar um cliente visual intuitivo (DBeaver) para enxergar as tabelas.

Passo 1: Subir o PostgreSQL em 1 comando no terminal

Terminal

Bash

```
docker run --name local-postgres -e POSTGRES_PASSWORD=postgres123 -e POSTGRES_DB=fintech_db -
```

Passo 2: Entendendo a String de Conexão (DATABASE_URL)

A string de conexão é o endereço unificado que seu código e ferramentas usam para logar no banco:

```
postgresql://postgres:postgres123@localhost:5432/fintech_db
```

- Nome do Banco (Database)
- Porta padrão do PostgreSQL
- Host / Endereço (máquina local)
- Senha de acesso
- Usuário administrador
- Protocolo do Driver

Passo 3: Conectar com o DBeaver (Ferramenta Gráfica Gratuita)

1. Baixe e instale o **DBeaver Community** (dbeaver.io).

2. Clique no ícone de **Nova Conexão** (tomada elétrica com um sinal de mais) e escolha **PostgreSQL**.
3. Preencha os campos com os dados da sua URL: Host: `localhost` , Porta: `5432` , Banco: `fintech_db` , Usuário: `postgres` , Senha: `postgres123` .
4. Clique em **Test Connection**. Aparecerá a mensagem "Conectado". Pronto! Agora você pode navegar pelas tabelas como se fossem pastas no seu computador.

7.5 Transações e Propriedades ACID em Sistemas Financeiros

Em pagamentos, um único clique do usuário gera múltiplas alterações em banco (ex: debitar R\$ 50 da carteira e criar um registro de pedido). Se a energia acabar no meio do caminho, o sistema não pode debitar o cliente sem registrar o pedido. É aqui que entram as propriedades **ACID**:

- **Atomicidade (All or Nothing):** Ou todas as operações acontecem com sucesso, ou nada acontece. Se houver falha, o comando `ROLLBACK` desfaz tudo.
- **Consistência:** Os dados devem respeitar as regras estabelecidas (ex: uma chave primária não pode ser duplicada, um saldo não pode ficar negativo se houver a regra `CHECK`).
- **Isolamento:** Transações simultâneas não interferem umas nas outras antes de serem finalizadas.
- **Durabilidade:** Uma vez finalizada com `COMMIT` , os dados estão gravados de forma permanente no disco.

Bloqueio Pessimista contra Double-Spending (SELECT FOR UPDATE)

Quando dois pagamentos concorrentes tentam gastar o mesmo saldo ao mesmo tempo, o backend utiliza o comando `SELECT ... FOR UPDATE` . Esse comando bloqueia temporariamente a linha daquele cliente no banco até que o cálculo termine, impedindo que duas requisições leiam o mesmo saldo no mesmo milissegundo.

7.6 Integrando o Banco de Dados nos Testes Playwright

Para auditar o banco dentro da sua suíte de automação com Playwright, instalamos a biblioteca oficial `pg` (node-postgres):

Terminal

Bash

```
npm install pg
npm install --save-dev @types/pg
```

Teste de Auditoria Contábil e Compliance PCI-DSS Automatizado

Veja como o Playwright dispara a requisição na API de pagamento e em seguida conecta diretamente no PostgreSQL para atestar a conformidade contábil e a segurança dos dados:

```
import { test, expect } from '../src/fixtures/apiFixtures';
import { PaymentFactory } from '../src/factories/PaymentFactory';

test.describe('Auditoria Automatizada de Banco de Dados e PCI-DSS', () => {
  test('deve criar transação na API e auditar persistência e compliance PCI no PostgreSQL', async () => {
    const paymentClient = new PaymentClient(db, merchantToken);
    // Pool PostgreSQL injetado automaticamente via Fixture!
    const amountInCents = 25000; // R$ 250,00
    const payload = PaymentFactory.createValidCreditCardPayload({ amountInCents });

    // 1. Ação: Realiza o pagamento via API HTTP
    const response = await paymentClient.createPayment(payload, { token: merchantToken });
    expect(response.status()).toBe(201);
    const { id } = await response.json();

    // 2. Auditoria Direta no Banco de Dados: Busca a linha gravada
    const queryResult = await db.query(
      `SELECT id, status, amount_in_cents, card_pan_masked, card_cvv, created_at
      FROM transactions
      WHERE id = $1`,
      [id] // Parameterized Query para evitar SQL Injection
    );

    // 3. Asserções de Validação de Banco
    expect(queryResult.rows.length).toBe(1);
    const row = queryResult.rows[0];

    // Validação Contábil (Centavos Exatos)
    expect(Number(row.amount_in_cents)).toBe(25000);
    expect(row.status).toBe('AUTHORIZED');

    // Validação Regulatória de Segurança PCI-DSS
    expect(row.card_pan_masked).toMatch(/^\\d{6}\\*{6}\\d{4}$/); // Ex: 411111*****1111
    expect(row.card_cvv).toBeNull(); // CVV NUNCA PODE SER PERSISTIDO
  });
});
```

8. Multi-Adquirência & Smart Routing com Fallback Resiliente

Gateways corporativos operam com roteamento inteligente entre múltiplas adquirentes (ex: Cielo, Stone, Rede). Se a adquirente primária retornar 504 Gateway Timeout ou erro interno, o sistema deve executar failover transparente para a secundária sem retornar erro ao cliente final.

tests/payments/smart-routing.spec.ts

TypeScript

```
import { test, expect } from '@playwright/test';

test('deve efetuar fallback automático para a adquirente secundária ao sofrer timeout na prim  
request,  
}) => {  
  // Configura mock para injetar timeout de 3.5s na Cielo  
  await request.post('/v1/mock-gateway/acquirers/cielo/configure', {  
    data: { behavior: 'GATEWAY_TIMEOUT', delayMs: 3500 },  
  });  
  
  const startTime = Date.now();  
  const chargeRes = await request.post('/v1/charges', {  
    data: {  
      amountInCents: 15000,  
      paymentMethod: 'CREDIT_CARD',  
      cardToken: 'tok_visa_valid_credit',  
    },  
    timeout: 8000,  
  });  
  
  const duration = Date.now() - startTime;  
  expect(chargeRes.status()).toBe(201);  
  const data = await chargeRes.json();  
  
  expect(data.status).toBe('AUTHORIZED');  
  expect(data.routing.primaryAcquirer).toBe('CIELO');  
  expect(data.routing.executedAcquirer).toBe('STONE');  
  expect(data.routing.fallbackReason).toBe('PRIMARY_ACQUIRER_TIMEOUT');  
  expect(duration).toBeGreaterThanOrEqual(3000);  
});
```

9. Validação de Contrato em Runtime com Zod

Interfaces TypeScript são removidas na compilação e não garantem segurança em runtime. Validações estritas com **Zod** asseguram que quebras de contrato (*breaking changes*) ou vazamento de campos confidenciais falhem a suíte imediatamente.

src/schemas/payment.schema.ts

TypeScript

```
import { z } from 'zod';

export const PaymentResponseSchema = z.object({
  id: z.string().uuid(),
  status: z.enum(['PENDING', 'AUTHORIZED', 'CAPTURED', 'FAILED', 'VOIDED', 'REFUNDED']),
  amountInCents: z.number().int().positive(),
  currency: z.literal('BRL'),
  authorizationCode: z.string().min(6).max(32).optional(),
  card: z.object({
    firstSixDigits: z.string().length(6),
    lastFourDigits: z.string().length(4),
    holderName: z.string(),
    expirationMonth: z.string(),
    expirationYear: z.string(),
  }).strict().optional(), // strict() impede vazamento de CVV ou dados adicionais
  createdAt: z.string().datetime(),
  updatedAt: z.string().datetime(),
}).strict();
```

10. Automação E2E de Pix (BACEN API v2: Cobrança e Devolução)

10.1 Padrões de Identificadores Regulamentados

- **txid:** Identificador criado pelo recebedor (26 a 35 caracteres alfanuméricos: `^[a-zA-Z0-9]{26,35}$`).
- **e2eid (End-to-End Identification):** Gerado exclusivamente pelo Banco Central/SPI na liquidação:

```
e2eid = "E" + ISPB (8 dígitos) + Data/Hora UTC (YYYYMMDDhhmm) + Sequencial (11 caracteres)
```

- **rtrId (Return Identification):** Gerado na devolução, iniciado por 'D'.

10.2 Emissão de Cobrança, Liquidação e Devolução Regulatória

```
import { test, expect } from '@playwright/test';
import crypto from 'node:crypto';

const txidRegex = /^[a-zA-Z0-9]{26,35}$/;
const e2eidRegex = /^E\d{8}\d{12}[a-zA-Z0-9]{11}$/;
const rtrIdRegex = /^D\d{8}\d{12}[a-zA-Z0-9]{11}$/;

test('ciclo completo Pix: emissão de cobrança imediata, liquidação SPI e devolução', async ({
  const txid = crypto.randomBytes(16).toString('hex');
  expect(txid).toMatch(txidRegex);

  // 1. Emissão da Cobrança
  const cobRes = await request.put(`/v2/cob/${txid}`, {
    data: {
      calendario: { expiracao: 3600 },
      valor: { original: '50.00' },
      chave: 'financeiro@empresa.com.br',
      solicitacaoPagador: 'Pedido #8821',
    },
  });
  expect(cobRes.status()).toBe(201);
  const { pixCopiaECola } = await cobRes.json();

  // 2. Simulação da Liquidação no Banco Central
  await request.post('/v2/sandbox/simulator/pay', {
    data: { pixCopiaECola },
  });

  // 3. Validação do e2eid retornado
  const checkRes = await request.get(`/v2/cob/${txid}`);
  const checkBody = await checkRes.json();
  expect(checkBody.status).toBe('CONCLUIDA');
  const e2eid = checkBody.pix[0].endToEndId;
  expect(e2eid).toMatch(e2eidRegex);

  // 4. Devolução Regulatória via BACEN v2
  const devId = crypto.randomBytes(8).toString('hex');
  const refundRes = await request.put(`/v2/pix/${e2eid}/devolucao/${devId}`, {
    data: {
      valor: '20.00',
      natureza: 'ORIGINAL',
      descricao: 'Devolucao parcial autorizada',
    },
  });
  expect([200, 201]).toContain(refundRes.status());
  const refundBody = await refundRes.json();
  expect(refundBody.id).toBe(devId);
  expect(refundBody.rtrId).toMatch(rtrIdRegex);
  expect(['EM_PROCESSAMENTO', 'DEVOLVIDO']).toContain(refundBody.status);
});
```

11. Webhooks: HMAC-SHA256, Desordenamento & Retentativas

11.1 Servidor Receptor Seguro com timingSafeEqual

Para atestar que o payload recebido não sofreu adulteração em trânsito, calculamos a assinatura HMAC-SHA256 sobre o buffer bruto e utilizamos `crypto.timingSafeEqual` para mitigar ataques de temporização (*Timing Attacks*).

```
import http, { createServer, Server, IncomingMessage, ServerResponse } from 'node:http';
import { AddressInfo } from 'node:net';
import { EventEmitter } from 'node:events';
import crypto from 'node:crypto';

export class SecureWebhookReceiver {
  private server: Server;
  private emitter = new EventEmitter();
  public port = 0;
  public url = '';

  constructor(private readonly secret: string) {
    this.server = createServer((req, res) => this.handle(req, res));
  }

  async start(): Promise<string> {
    return new Promise((resolve) => {
      this.server.listen(0, '127.0.0.1', () => {
        const addr = this.server.address() as AddressInfo;
        this.port = addr.port;
        this.url = `http://127.0.0.1:${this.port}/webhook`;
        resolve(this.url);
      });
    });
  }

  private handle(req: IncomingMessage, res: ServerResponse): void {
    const chunks: Buffer[] = [];
    req.on('data', (c) => chunks.push(c));
    req.on('end', () => {
      const rawBody = Buffer.concat(chunks);
      const signature = req.headers['x-signature-sha256'] as string;

      if (!signature) {
        res.writeHead(401).end(JSON.stringify({ error: 'Assinatura ausente' }));
        return;
      }

      const computed = crypto
        .createHmac('sha256', this.secret)
        .update(rawBody)
        .digest('hex');

      const bufExpected = Buffer.from(signature, 'hex');
      const bufComputed = Buffer.from(computed, 'hex');

      if (bufExpected.length !== bufComputed.length || !crypto.timingSafeEqual(bufExpected, bufComputed)) {
        res.writeHead(403).end(JSON.stringify({ error: 'Assinatura invalida' }));
        return;
      }

      const payload = JSON.parse(rawBody.toString('utf-8'));
      this.emitter.emit(`evt:${payload.id}`, payload);
      res.writeHead(200).end(JSON.stringify({ status: 'DELIVERED' }));
    });
  }
}
```

```

    async stop(): Promise<void> {
      return new Promise((resolve) => this.server.close(() => resolve()));
    }
  }
}

```

11.2 Entrega Fora de Ordem (Out-of-Order Webhooks)

Em sistemas assíncronos que utilizam mensageria (Kafka, RabbitMQ, AWS SQS), a ordem de entrega não é garantida. O webhook receptor deve tolerar a chegada do evento `PAYMENT_CAPTURED` antes de `PAYMENT_AUTHORIZED` sem regredir o estado do pedido.

tests/webhooks/out-of-order.spec.ts

TypeScript

```

import { test, expect } from '@playwright/test';

test('o receptor não deve regredir status se AUTHORIZED chegar após CAPTURED', async ({ request }) {
  const orderId = `order_${Date.now()}`;

  // 1. Dispara evento de CAPTURA (adiantado)
  const capRes = await request.post('/api/webhooks/listener', {
    data: {
      id: `evt_cap_${Date.now()}`,
      type: 'PAYMENT_CAPTURED',
      data: { orderId, status: 'CAPTURED', timestamp: Date.now() },
    },
  });
  expect(capRes.status()).toBe(200);

  // 2. Dispara evento de AUTORIZAÇÃO (atrasado)
  const authRes = await request.post('/api/webhooks/listener', {
    data: {
      id: `evt_auth_${Date.now()}`,
      type: 'PAYMENT_AUTHORIZED',
      data: { orderId, status: 'AUTHORIZED', timestamp: Date.now() - 5000 },
    },
  });
  expect(authRes.status()).toBe(200);

  // 3. Validação do estado final na aplicação
  const orderRes = await request.get(`/v1/orders/${orderId}`);
  const order = await orderRes.json();
  expect(order.status).toBe('CAPTURED');
});

```

12. Timeouts, Quedas de Conexão & Engenharia de Caos (Toxiproxy)

O **Toxiproxy** (Shopify) atua como proxy TCP intermediário, permitindo que a suíte Playwright altere condições de rede em tempo de execução para atestar a robustez do sistema frente a panes reais.

12.1 Orquestração com Docker Compose

docker-compose.yml

YAML

```
version: '3.8'
services:
  toxiproxy:
    image: ghcr.io/shopify/toxiproxy:2.11.0
    container_name: toxiproxy
    ports:
      - "8474:8474"           # API Administrativa
      - "8580-8590:8580-8590" # Portas alocadas por worker Playwright
    networks:
      - chaos-net

  mock-acquirer:
    image: wiremock/wiremock:3.5.2
    container_name: mock-acquirer
    ports:
      - "8080:8080"
    networks:
      - chaos-net

networks:
  chaos-net:
    driver: bridge
```

12.2 Injeção de Latência com Assert de Timeout no Playwright

```
import { test, expect } from '@playwright/test';

test('deve abortar requisição por timeout de 2s ao injetar 3500ms de latência no Toxiproxy',
  request,
) => {
  // 1. Injeta toxic de latência na porta 8474 do Toxiproxy
  await request.post('http://localhost:8474/proxies/acquirer_proxy/toxics', {
    data: {
      name: 'latency_downstream',
      type: 'latency',
      stream: 'downstream',
      attributes: { latency: 3500, jitter: 50 },
    },
  });

  const startTime = Date.now();
  let timedOut = false;

  try {
    await request.post('http://localhost:8580/v1/charges', {
      data: { amountInCents: 10000 },
      timeout: 2000, // Limite máximo aceito
    });
  } catch (err: any) {
    timedOut = true;
    expect(err.message).toMatch(/timed out after 2000ms/);
  }

  const duration = Date.now() - startTime;
  expect(timedOut).toBe(true);
  expect(duration).toBeGreaterThanOrEqual(1950);

  // Limpeza do Toxic
  await request.delete('http://localhost:8474/proxies/acquirer_proxy/toxics/latency_downstream');
};
```

13. Pipeline de CI/CD no GitHub Actions

13.1 Pipeline de Execução Contínua

.github/workflows/api-ci.yml

YAML

```
name: API Automation Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15

    env:
      API_BASE_URL: ${ secrets.API_BASE_URL }
      CLIENT_ID: ${ secrets.CLIENT_ID }
      CLIENT_SECRET: ${ secrets.CLIENT_SECRET }
      DATABASE_URL: ${ secrets.DATABASE_URL }

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Node.js 20
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install Dependencies
        run: npm ci

      - name: Typecheck Estrito & Lint
        run: |
          npm run lint
          npx tsc --noEmit

      - name: Execute Playwright API Tests
        run: npx playwright test --workers=4

      - name: Upload Artifacts on Failure
        if: failure()
        uses: actions/upload-artifact@v4
        with:
          name: playwright-report
          path: playwright-report/
          retention-days: 14
```

14. Kit de Sobrevivência Diário: Debug, Observabilidade & Banco Seguro

Na rotina real de um QA Pleno em pagamentos, o console de testes locais é apenas o ponto de partida. Quando um bug ocorre em homologação ou produção, o erro raramente é evidente. Este capítulo entrega os métodos exatos de investigação e manipulação segura de dados no dia a dia.

14.1 Observabilidade em Produção: Investigando Falhas Reais

Quando um teste automatizado ou manual quebra, o time de engenharia não quer saber apenas que "falhou com 500". Você deve entregar o rastro exato do incidente utilizando ferramentas como Datadog, Kibana (OpenSearch) ou AWS CloudWatch.

A. Como rastrear usando Correlation ID e Trace ID

1. Ao disparar um teste, copie o valor do cabeçalho `X-Correlation-ID` enviado ou o `id` da transação retornado.
2. Abra a ferramenta de logs (ex: Datadog / Kibana) e filtre pelo campo indexado:

Kibana / Datadog Search Filter

Query

```
correlation_id: "7a9e34b1-8409-42b8-9311-bfad6ce2e104" OR transaction_id: "d3b07384-d113-4
```

3. Você verá a árvore completa de microsserviços por onde a requisição passou: *API Gateway* → *Microsserviço de Autorização* → *Motor Antifraude* → *Adquirente Externa*.
4. Identifique onde a linha do log passou de `INFO` para `ERROR` ou `WARN`.

14.2 De-Para dos Códigos de Retorno das Adquirentes

Quando a adquirente recusa uma transação, ela devolve um código bruto numérico. O gateway mapeia esse código para uma mensagem interna. Tenha esta tabela na sua mesa de trabalho para diagnosticar se a falha é do sistema ou da regra do cartão:

Código Bruto (Cielo / Rede / Stone)	Significado Oficial da Bandeira	Comportamento Esperado da API	Ação do QA
00 ou 000	Transação Aprovada com Sucesso.	Status 201 Created , estado AUTHORIZED .	Caminho feliz validado.
05	Não Autorizada / Recusa Genérica pelo Banco Emissor.	Status 422 , código TRANSACTION_DENIED .	Validar se o motivo amigável não vaza dados sensíveis ao pagador.
51	Saldo Insuficiente (Limite estourado).	Status 422 , código INSUFFICIENT_FUNDS .	Validar tratamento do erro sem reproprocessamento indevido.
57	Cartão Expirado ou Inválido para essa Operação.	Status 422 , código EXPIRED_CARD .	Verificar se a validação prévia de DTO barrou antes de chamar a ponta.
99 ou 999	Timeout de Infraestrutura / Adquirente Indisponível.	Status 504 Gateway Timeout ou acionamento de Fallback.	Verificar se o Smart Routing chaveou para a adquirente secundária.

14.3 Diretrizes de Segurança em Banco de Dados no Dia a Dia

Como QA Pleno, você terá acesso a bancos de Staging (Homologação) e réplicas de Produção. Um comando errado pode parar a operação da empresa ou apagar registros contábeis. Siga rigorosamente estas regras:

Regra de Ouro: Nunca execute UPDATE ou DELETE direto

Em qualquer ambiente compartilhado, **sempre** abra uma transação manual explícita antes de rodar comandos de alteração. Isso permite inspecionar quantas linhas foram afetadas antes de confirmar.

Como alterar ou limpar dados de teste com segurança absoluta:

```
-- 1. Inicie a transação manual
BEGIN;

-- 2. Execute a alteração desejada
UPDATE transactions
SET status = 'VOIDED'
WHERE idempotency_key = 'test-replay-uuid-881';

-- 3. Verifique o resultado
SELECT id, status, amount_in_cents
FROM transactions
WHERE idempotency_key = 'test-replay-uuid-881';

-- 4A. Se alterou APENAS o registro correto:
COMMIT; -- Grava a alteração definitivamente

-- 4B. Se você esqueceu o WHERE ou alterou mais de 1 linha por engano:
ROLLBACK; -- Desfaz TUDO imediatamente como se nada tivesse acontecido!
```

- **Use Sempre a Read Replica:** Para relatórios, extração de massa ou investigação de bugs, nunca conecte no banco principal de escrita (Master/Writer). Conecte na réplica de leitura (Reader) para não onerar o pool de transações dos clientes reais.
- **Evite Lock de Tabelas:** Consultas com `SELECT * FROM transactions` sem `LIMIT` em tabelas com milhões de linhas geram consumo excessivo de memória e travam threads no banco. Use sempre `LIMIT 50` e filtre por índices (ex: `created_at ≥ CURRENT_DATE`).

15. Auditoria de PRs, Gestão de Massa & Plano Tático de 60 Dias

15.1 O que Auditar nos Pull Requests (PRs) dos Desenvolvedores

Um QA Pleno atua na prevenção de defeitos antes do merge em `develop` ou `main`. Ao ser marcado como revisor no GitHub/GitLab, busque ativamente por estas quatro vulnerabilidades recorrentes:

Ponto Crítico no Código	O que Procurar nos Arquivos Alterados	Risco Real em Produção	Comentário que Você Deve Fazer no PR
Precisão de Moeda	Variáveis tipadas como <code>number</code> no TypeScript ou <code>float/double</code> recebendo valores com vírgula (ex: <code>amount = 10.50</code>).	Quebra da norma IEEE 754 gerando centavos fantasmas e discrepâncias contábeis no fechamento do dia.	"Favor refatorar para utilizar valores inteiros em centavos (<code>amountInCents</code>) ou tipo <code>Decimal</code> exato para evitar erros de ponto flutuante."
Concorrência / Saldo	Métodos de débito de carteira fazendo apenas <code>SELECT saldo</code> seguido de <code>UPDATE saldo</code> sem lock.	Vulnerabilidade a <code>Double-Spending</code> via requisições simultâneas no mesmo milissegundo.	"Esta operação de débito precisa de lock pessimista (<code>SELECT FOR UPDATE</code>) ou lock distribuído via Redis para evitar race conditions."
Timeouts em Clientes HTTP	Instâncias de <code>axios</code> , <code>fetch</code> ou <code>got</code> chamando serviços externos sem propriedade <code>timeout</code> definida.	Se a adquirente sofrer instabilidade, as conexões ficam presas indefinidamente até derrubar os containers da API.	"Chamada externa sem timeout explícito configurado. Configurar timeout máximo de 5000ms com política de contingência."
Logs e Segurança PCI-DSS	Comandos <code>logger.info(req.body)</code> ou <code>console.log</code> gravando payloads brutos de entrada.	Vazamento de CVV ou PAN sem máscara nos logs do Datadog/CloudWatch, gerando não conformidade PCI e multas.	"O payload impresso nos logs contém dados abertos do cartão. Aplicar sanitização/mascaramento antes de registrar o log."

15.2 Pirâmide de Execução de Testes e Gestão de Massa

Para manter a suite de testes rápida e sem falsos-positivos (*flaky tests*), estruture a bateria em duas camadas distintas:

CAMADA 1: Testes de PR (Fast Feedback - 2 a 4 minutos)

- Executados a cada commit / Pull Request no GitHub Actions
- Dependências externas 100% mockadas (WireMock ou Mocks locais em Node.js)
- Foco: Contratos (Zod), status codes, validações 400/422, cálculos de Split e regras de negócio

CAMADA 2: Testes E2E de Staging / Homologação (15 a 30 minutos)

- Executados a cada deploy no ambiente de staging ou agendados de madrugada (Nightly)
- Comunicação real com Sandboxes de Adquirentes (Cielo, Stone) e SPI do Banco Central
- Foco: Transições de estado completas, reconciliação com PostgreSQL e latência real

Gestão Saudável de Massa de Testes

- **Geração Algorítmica de Cartões:** Nunca utilize números estáticos em planilhas compartilhadas. Utilize geradores com o Algoritmo de Luhn que forneçam BINs aceitos pelo sandbox configurado.
- **Teardown e Limpeza:** Suas fixtures de teste de integração devem conter blocos de cancelamento (`afterEach` ou rotina dedicada) para cancelar pré-autorizações e expirar cobranças Pix abertas, evitando poluição visual nos dashboards compartilhados de homologação.

15.3 Curso Prático: Roadmap e Evolução do QA Pleno

Com base em feedbacks de estruturação, a abordagem de aprendizado não deve ser assimilar todas as tecnologias simultaneamente. O plano abaixo refina os 60 dias em um roadmap lógico de 8 níveis, garantindo base sólida antes de avançar para arquiteturas complexas.

★ Etapa Permanente — Test Design

Antes de automatizar, você deve saber quais testes deveriam existir. Para cada endpoint (ex: `POST /payments`), crie uma matriz contemplando: Happy path, Boundary, Negative, Auth, Authorization, State, Idempotency, Concurrency, Contract, Database, Security, Resilience e Integration. Só depois você automatiza.

Nível	Foco do Aprendizado	Habilidades & Ferramentas Essenciais
 NÍVEL 1	Base Técnica	TypeScript, async/await, Promise, HTTP, JSON, REST, headers, status codes, autenticação e Git. <i>(Sem isso, não avance).</i>
 NÍVEL 2	Playwright API	Domínio dos métodos (<code>get</code> , <code>post</code> , etc.), assertions, fixtures, variáveis de ambiente, retries, parallelism e reports.
 NÍVEL 3	Arquitetura	Design Patterns: <code>Service</code> <code>Object</code> → <code>Fixture</code> → <code>Factory</code> → <code>DTO</code> → <code>Schema</code> . Saber responder por que uma classe existe e como evitar duplicação.
 NÍVEL 4	API Testing Avançado	Comportamento esperado por status (não apenas 200), boundary testing, transições de estado, idempotência e testes de fluxo (ex: Criar → Capturar → Estornar).
 NÍVEL 5	Banco de Dados	PostgreSQL: Operações básicas (SELECT, JOIN), constraints, transactions, ACID. Garantir que a API retornou 201 e o banco está consistente.
 NÍVEL 6	Contract Testing	Zod, JSON Schema e OpenAPI. Compreender a diferença entre validação TypeScript (compilação) e runtime.
 NÍVEL 7	CI/CD	GitHub Actions: Pull Request, Lint, Typecheck, API Tests, Reports, Artifacts e gestão de Secrets.
 NÍVEL 8	Complexidades Reais	Webhooks (HMAC, fora de ordem), Resiliência (Toxiproxy, retries), Docker, Observabilidade (Logs, Trace ID, Datadog) e Segurança (OWASP API Top 10).